Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka Techniczna

Implementacja i analiza efektywności algorytmu losowego, zachłannego - Nearest Neighour (NN) i Repetitive Nearest Neighbour (RNN) oraz Brute-Force (przegląd zupełny)

Imię i nazwisko: Wiktor Zięba
Numer albumu: 281056

Wrocław, marzec 2026

Spis treści

1 Wstęp teoretyczny **2**

2 Opis testowanych algorytmów **2**
2.1 Algorytm Brute Force 2
2.2 Algorytm Nearest Neigbour 3
2.3 Algorytm Repetitive Nearest Neigbour 4
2.4 Algorytm losowy 5

3 Plan eksperymentu **5**
3.1 Cel eksperymentu 5
3.2 Pomiar czasu wykonania Brute Force 6
3.3 Porównanie błędów względnych rozwiązań algorytmów 6

4 Wyniki eksperymentów **7**
4.1 Wyniki dla algorytmu Brute Force 7
4.2 Porównanie błędów względnych algorytmów NN, RNN oraz losowego 8

5 Wnioski **10**

6 Literatura **11**

1 Wstęp teoretyczny

Problem komiwojażera to zagadnienie optymalizacyjne polegające na znalezieniu w pełnym grafie ważonym najkrótszej ścieżki począwszy od punktu startowego grafu która przechodzi przez wszystkie inne wierzchołki dokładnie raz i kończy się w punkcie startowym (cykl Hamiltona). W przypadku asymetrycznego problemu komiwojażera, który jest przedmiotem badania koszt przejścia z wierzchołka X do wierzchołka Y może mieć różną wartość od kosztu przejścia z wierzchołka Y do X. Istnieją dwa podstawowe podejścia rozwiązywania problemu komiwojażera:

- **Algorytmy dokładne** – wyznaczają rozwiązanie optymalne, jednak ich złożoność czasowa rośnie bardzo szybko wraz ze wzrostem liczby wierzchołków co sprawia, że dla większych instacji problemu ich zastosowanie staje się niepraktyczne
- **Algorytmy heurystyczne** - algorytmy pozwalające uzyskać wynik w znacznie krótszym czasie kosztem tego, że nie gwarantują znalezienia rozwiązania optymalnego

Problem komiwojażera należy do klasy problemów NP-trudnych, nie jest znany algorytm wielomianowy znajdujący optymalne rozwiązanie dla każdego przypadku. Wraz ze wzrostem ilości miast, liczba możliwych tras rośnie silniowo dla metod opartych na pełnym przeglądzie, co czyni obliczenie optymalnej trasy niemożliwym dla dużych zbiorów danych.

2 Opis testowanych algorytmów

W ramach wykonanego projektu zaimplementowano cztery algorytmy służące do rozwiązywania asymetrycznego problemu komiwojażera: algorytm Brute Force, algorytm Nearest Neighbour (NN), algorytm Repetitive Nearest Neigbour (RNN) oraz algorytm losowy.

2.1 Algorytm Brute Force

Algorytm Brute Force to algorytm dokładny. Polega na wygenerowaniu wszystkich możliwych permutacji odwiedzania wierzchołków, a następnie obliczeniu kosztu każdej otrzymanej trasy. Na końcu zostaje wybrana trasa o najmniejszym obliczonym koszcie. Złożoność czasowa algorytmu szacuje się na O(n!) ponieważ konieczne jest sprawdzenie wszystkich możliwych permutacji odwiedzenia wierzchołków.

```
BRUTE_FORCE(M, start)

    // Ustawienie wartosci poczatkowych
    najlepszyKoszt = INF
    najlepszaTrasa = pusty ciag
    V = zbior wszystkich wierzcholkow z wyjatkiem start

    // Sprawdzenie kazdej mozliwej kolejnosci wierzcholkow
    DLA kazdej permutacji P zbioru V WYKONAJ
        koszt = 0
        poprzedni = start

        // Obliczenie kosztu przejscia zgodnie z aktualna permutacja
        DLA kazdego wierzcholka v w P WYKONAJ
            koszt = koszt + kosztPrzejscia(poprzedni, v)
            poprzedni = v

        // Dodanie kosztu powrotu do wierzcholka startowego
        koszt = koszt + kosztPrzejscia(poprzedni, start)

        // Sprawdzenie, czy znaleziona trasa jest lepsza
        JEZELI koszt < najlepszyKoszt:
```

```
            najlepszyKoszt = koszt
            najlepszaTrasa = [start] + P + [start]

    ZWROC (najlepszaTrasa, najlepszyKoszt)
```

Tekst 1: Pseudokod algorytmu Brute Force

1. Zalety:
   - Gwarancja wyznaczenia rozwiązania optymalnego
2. Wady:
   - Bardzo wysoka złożoność czasowa O(n!)
   - Znajduje zastosowanie tylko dla małych instancji problemu

2.2 Algorytm Nearest Neigbour

Algorytm NN jest algorytmem heurystycznym. Algorytm startuje w punkcie początkowym a następnie dla każdej iteracji wybiera jeszcze nieodwiedzony wierzchołek, do którego koszt przejścia jest najmniejszy i oznacza go jako odwiedzony. W zaimplementowanej wersji w momencie gdy istnieje kilka przejść o tym samym koszcie, algorytm rekurencyjnie sprawdza każdą możliwą opcje a na koniec wybiera trasę o najmniejszym koszcie. Proces jest powtarzany aż do odwiedzenia wszystkich wierzchołków. Algorytm podejmuje w każdym kroku decyzję najbardziej optymalną lokalnie poprzez wybór najkrótszej drogi, co jednak nie gwarantuje optymalnego rozwiązania globalnie. Dzięki zaimplementowaniu rekurencyjnego sprawdzania ścieżek maleje szansa na wyszukanie nieoptymalnej ścieżki przez algorytm lecz wciąż nie gwarantuje optymalnego rozwiązania. Złożoność czasowa algorytmu wynosi $O(n^2)$ ponieważ w każdym kroku algorytm przeszukuje wszystkie nieodwiedzone wierzchołki w celu wybrania najtańszego przejścia. Z uwagi na zaimplementowanie rekurencyjnego rozwiązywania remisów ilość powtarzających się sytuacji remisowych może znacząco spowolnić wykonywanie się algorytmu, w momencie gdy w macierz wypełniona jest pojedynczą wartością złożonosć spada do O(n!).

```
NEAREST_NEIGHBOUR(M, start)

    najlepszyKoszt = INF
    najlepszaTrasa = pusty ciag

    SZUKAJ(aktualny, odwiedzone, trasa, kosztDotychczasowy)

        JEZELI liczba elementow w odwiedzone = liczba wszystkich wierzcholkow:
            kosztKoncowy = kosztDotychczasowy + kosztPrzejscia(aktualny, start
                )
            trasaKoncowa = trasa + [start]

            JEZELI kosztKoncowy < najlepszyKoszt:
                najlepszyKoszt = kosztKoncowy
                najlepszaTrasa = trasaKoncowa

            ZAKONCZ

        minimalnyKoszt = INF

        DLA kazdego wierzcholka v WYKONAJ
            JEZELI v nie nalezy do odwiedzone ORAZ kosztPrzejscia(aktualny, v)
                 < minimalnyKoszt:
                minimalnyKoszt = kosztPrzejscia(aktualny, v)

        DLA kazdego wierzcholka v WYKONAJ
```

```
            JEZELI v nie nalezy do odwiedzone ORAZ kosztPrzejscia(aktualny, v)
                = minimalnyKoszt:
                noweOdwiedzone = odwiedzone + v
                nowaTrasa = trasa + v
                nowyKoszt = kosztDotychczasowy + kosztPrzejscia(aktualny, v)

                SZUKAJ(v, noweOdwiedzone, nowaTrasa, nowyKoszt)

    SZUKAJ(start, {start}, [start], 0)

    ZWROC (najlepszaTrasa, najlepszyKoszt)
```

Tekst 2: Pseudokod algorytmu Nearest Neighbour

1. Zalety:
   - Szybkie uzyskanie rozwiązania nawet dla dużych instancji problemu
2. Wady:
   - Brak gwarancji znalezienia rozwiązania optymalnego
   - Ryzyko uzyskania słabego wyniku końcowego

2.3 Algorytm Repetitive Nearest Neigbour

Algorytm RNN to rozwiązanie heurystyczne stanowiące rozszerzenie klasycznego algorytmu Nearest Neigbour. Polega na wielokrotnym uruchomieniu algorytmu NN rozpoczynając z każdego możliwego wierzchołka startowego. Następnie wyłania najlepszą trasę o najmniejszym koszcie. Podejście to zmniejsza zależność wyniku od wyboru punktu startowego i umożliwia uzyskanie lepszego wyniku niż przy pomocy algorytmu NN. Złożoność czasowa algorytmu RNN wynosi $O(n^3)$ ponieważ algorytm NN o złożoności $O(n^2)$ uruchamiany jest dla każdego z n możliwych wierzchołków startowych. W zastosowanej implementacji, przy wielu remisach realnie złożoność może być zbliżona do rzędu O(n*n!).

```
RNN(M, start)

    najlepszyWynik = NEAREST_NEIGHBOUR(M, start)

    DLA kazdego wierzcholka s WYKONAJ
        JEZELI s != start:
            aktualnyWynik = NEAREST_NEIGHBOUR(M, s)

            JEZELI aktualnyWynik.koszt < najlepszyWynik.koszt:
                najlepszyWynik = aktualnyWynik

    ZWROC najlepszyWynik
```

Tekst 3: Pseudokod algorytmu Repetitive Nearest Neighbour

1. Zalety:
   - Możliwe uzyskanie lepszego rozwiązania niż w klasycznym NN
   - Mniejsza zależność wyniku od wyboru wierzchołka początkowego
2. Wady:
   - Brak gwarancji znalezienia rozwiązania optymalnego
   - Dłuższy czas działania niż algoprytm NN

2.4 Algorytm losowy

Algorytm losowy to algorytm heurystyczny. Wielokrotnie generuje losowe permutacje odwiedzania wierzchołków, a następnie oblicza koszt każdej trasy. Na koniec wybiera trasę o najmniejszym koszcie. Algorytm losowy różni się od metody Brute force tym, że analizuje jedynie ustaloną ilość losowych permutacji zamiast przeglądać wszystkie możliwe. Dzięki temu czas działania jest znacznie krótszy od przeglądu całkowitego. Jakość uzyskanego wyniku jest zależna od ustalonej ilości permutacji. Złożoność czasowa algorytmu wynosi O(p*n) gdzie p oznacza ilość permutacji. Wynika to z faktu, że dla każdej wygenerowanej trasy konieczne jest przejście po wszystkich wierzchołkach. W przypadku algorytmu losowego można ograniczyć możliwość wielokrotnego wygenerowania tej samej permutacji zapamiętując wcześniej wygenerowane permutacje i odrzucając te, które już wystąpiły. Wadą takiego podejścia dodatkowy koszt sprawdzania, czy dana permutacja została już wcześniej wygenerowana.

```
ALGORYTM_LOSOWY(M, start, liczbaPermutacji)

    najlepszyKoszt = INF
    najlepszaTrasa = pusty ciag
    V = zbior wszystkich wierzcholkow z wyjatkiem start

    // Wielokrotne losowanie tras
    DLA i = 1 DO liczbaPermutacji WYKONAJ
        P = losowa permutacja zbioru V
        koszt = 0
        poprzedni = start

        // Obliczenie kosztu dla wylosowanej trasy
        DLA kazdego wierzcholka v w P WYKONAJ
            koszt = koszt + kosztPrzejscia(poprzedni, v)
            poprzedni = v

        // Dodanie kosztu powrotu do wierzcholka startowego
        koszt = koszt + kosztPrzejscia(poprzedni, start)

        // Sprawdzenie, czy nowa trasa jest lepsza
        JEZELI koszt < najlepszyKoszt:
            najlepszyKoszt = koszt
            najlepszaTrasa = start + P + start

    ZWROC (najlepszaTrasa, najlepszyKoszt)
```

Tekst 4: Pseudokod algorytmu losowego

1. Zalety:
   - Krótki czas działania
   - Możliwość poprawienia jakości wyniku poprzez zwiększenie ilości permutacji
2. Wady:
   - Brak gwarancji znalezienia rozwiązania optymalnego
   - Dla małej liczby prób może zwracać słaby wynik

3 Plan eksperymentu

3.1 Cel eksperymentu

Celem eksperymentu było porównanie czasu wykonania algorytmu Brute Force w zależności od ilości miast (wielkości macierzy) oraz wyliczenie i porównanie błędu uzyskiwanego przez pozostałe algorytmy

w stosunku do przeglądu zupełnego.
Instancja problemu była reprezentowana w postaci macierzy o rozmairze N x N. Wartość macierzy w każdym punkcie [i][j] oznaczała koszt przejścia z wierzchołka i do wierzchołka j. Wszystkie koszty były wartościami całkowitymi, w testach generowano dla każdej macierzy wartośći zakresu od 1 do 100. Na przekątnej macierzy (koszt przejścia z tego samego wierzchołka) umieszczono wartości -1.
Program umożliwia zarówno wczytanie danych macierzy z pliku jak i generowanie losowych danych dla podanej wielkości macierzy oraz wartości maksymalnej i minimalnej.

3.2 Pomiar czasu wykonania Brute Force

Pomiary dla algorytmu Brute Force wykonano dla siedmiu różnych rozmiarów macierzy:
$N \in \{8, 9, 10, 11, 12, 13, 14\}$ wyznaczając dla każdego rozmiaru czas potrzebny na znalezienie rozwiązania przez algorytm. Pomiar czasu uruchamiano bezpośrednio przed uruchomieniem i kończono bezpośrednio po jego zakończeniu sie. W pomiarze uwzględniano wyłącznie czas działania algorytmu bez czasu potrzebnego na np. generowanie danych. Do pomiarów użyto `std::chrono::high_resolution_clock` zliczając czas mikrosekundach a następnie przeliczając go na milisekundy.

3.3 Porównanie błędów względnych rozwiązań algorytmów

Porównanie algorytmów Nearest Neighbour, Repetitive Nearest Neighbour oraz algorytmu losowego przeprowadzono na podstawie błędu względnego rozwiązania. Jako punkt odniesienia przyjęto rozwiązanie optymalne wyznaczone za pomocą algorytmu Brute Force.
Badania wykonano dla pięciu różnych rozmiarów macierzy: $N \in \{10, 11, 12, 13, 14\}$. Dla każdego N wygenerowano 100 losowych instancji problemu. Dla każdej instancji wyznaczano koszt rozwiązania optymalnego za pomocą algorytmu Brute Force, a następnie uruchamiano pozostałe badane algorytmy i zapisywano koszt uzyskanej przez nie trasy.
W przypadku algorytmu losowego liczba analizowanych permutacji była równa $10N$, zgodnie z wymaganiami projektu.
Jako miarę jakości rozwiązania przyjęto błąd względny, obliczany według wzoru:

$$\delta = \frac{Koszt_{alg} - Koszt_{opt}}{Koszt_{opt}} \cdot 100\%$$

gdzie $Koszt_{alg}$ oznacza koszt trasy wyznaczonej przez badany algorytm, natomiast $Koszt_{opt}$ oznacza koszt trasy optymalnej wyznaczonej za pomocą algorytmu BF.
Dla każdej wartości N i dla każdego algorytmu obliczono następnie średni błąd względny na podstawie 100 pomiarów.

4 Wyniki eksperymentów

4.1 Wyniki dla algorytmu Brute Force

W tabeli 1 przedstawiono czasy wykonania algorytmu Brute Force dla coraz to większych wartości N. Na podstawie otrzymanych danych widoczny jest gwałtowny wzrost czasu działania wraz ze wzrostem liczby miast. Szczególnie widoczy skok wystąpił dla $N = 14$, gdzie czas wykonania wyniósł 230176,967 ms (230,18 s), również dla tej wartości N czas osiągnął wartość najbardziej zbliżoną czasowi 2 minut (3 minuty i 50 sekund.). Dla wartości $N = 15$ nie uzyskano wyniku, ponieważ czas działania algorytmu okazał się zbyt długi.

Table 1: Czas wykonania algorytmu Brute Force

N	Czas [ms]
8	0,460
9	3,762
10	40,085
11	438,074
12	1713,107
13	16340,561
14	230176,967

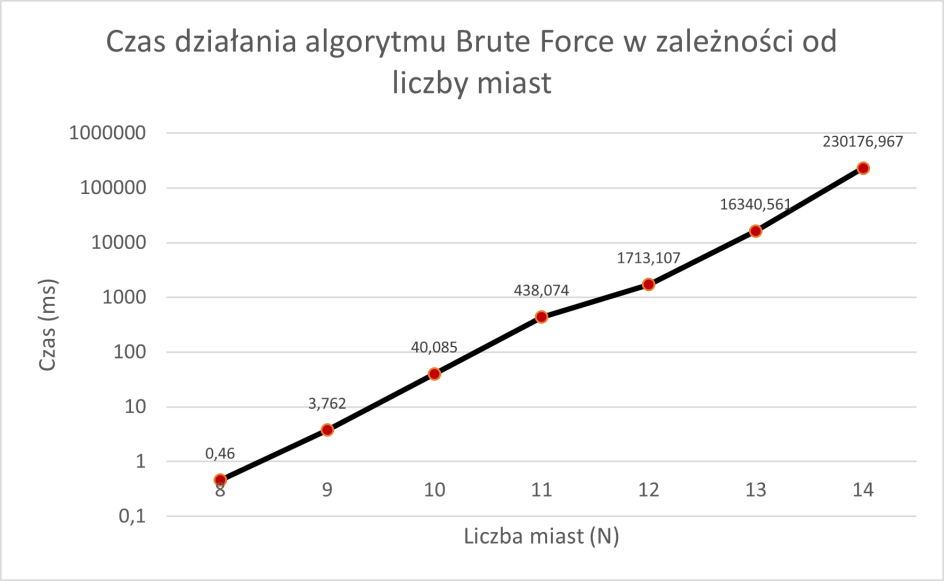

Figure 1: Zależność czasu działania algorytmu Brute Force od liczby miast

4.2 Porównanie błędów względnych algorytmów NN, RNN oraz losowego

Dla algorytmów Nearest Neighbour, Repetitive Nearest Neighbour oraz algorytmu losowego obliczono średni błąd względny na podstawie 100 wygenerowanych instancji dla każdej wartości $N \in \{10, 11, 12, 13, 14\}$.

Table 2: Średni błąd względny algorytmu NN

N	NN [%]
10	52,93
11	59,02
12	61,56
13	55,18
14	67,49

Table 3: Średni błąd względny algorytmu RNN

N	RNN [%]
10	17,32
11	17,48
12	19,52
13	19,43
14	23,63

Table 4: Średni błąd względny algorytmu losowego

N	Losowy [%]
10	95,04
11	105,64
12	129,66
13	149,17
14	170,64

Z zebranych danych wynika, że spośród badanych algorytmów heurystycznych najlepsze rezultaty uzyskał algorytm Repetitive Nearest Neighbour. Dla każdej badanej wartości N osiągał on najniższy średni błąd względny. Algorytm Nearest Neighbour dawał wyniki nieco gorsze, jednak różnice między NN i RNN nie były tak drastyczne jak w porównaniu do algorytmu losowego, którego średni błąd względny był wyraźnie większy. Można również zauważyć, że wraz ze wzrostem rozmiaru problemu rosła wartość średniego błędu względnego dla wszystkich z badanych algorytmów heurystycznych. Najwiekszy wzrost odnotował algorytm losowy, co wskazuje, że przy liczbie permutacji równej $10N$ jego skuteczność staje się niewystarczająca dla coraz t o większych wartości N.

Na wykresie 5 przedstawiono wspólne porównanie średnich błędów względnych wszystkich badanych algorytmów heurystycznych.

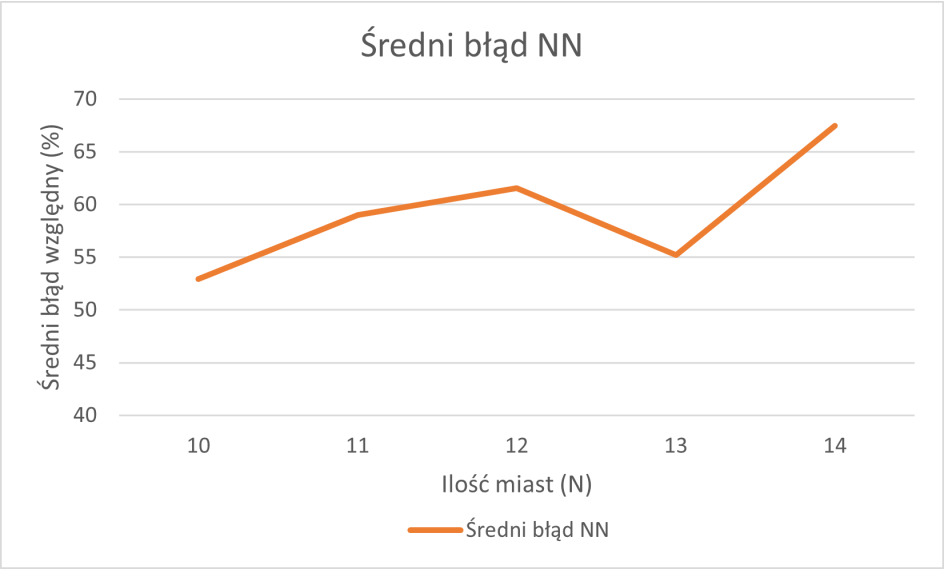

Figure 2: Średni błąd algorytmu NN w zależności od rozmiaru N

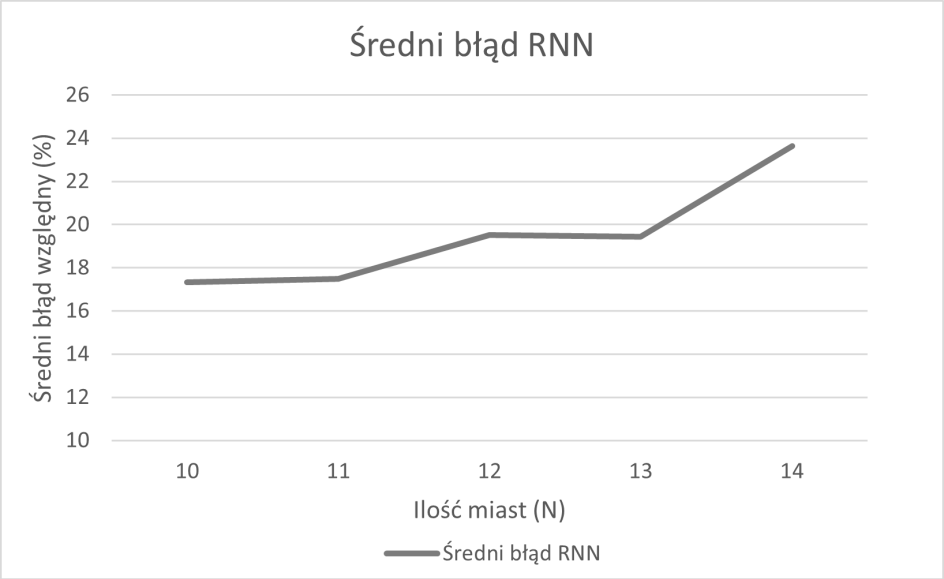

Figure 3: Średni błąd algorytmu RNN w zależności od rozmiaru N

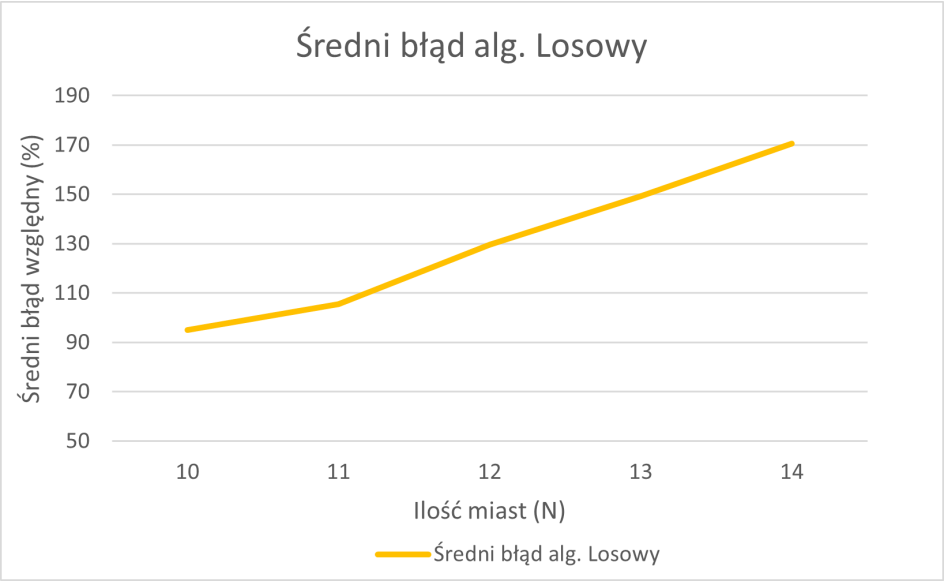

Figure 4: Średni błąd algorytmu losowego w zależności od rozmiaru N

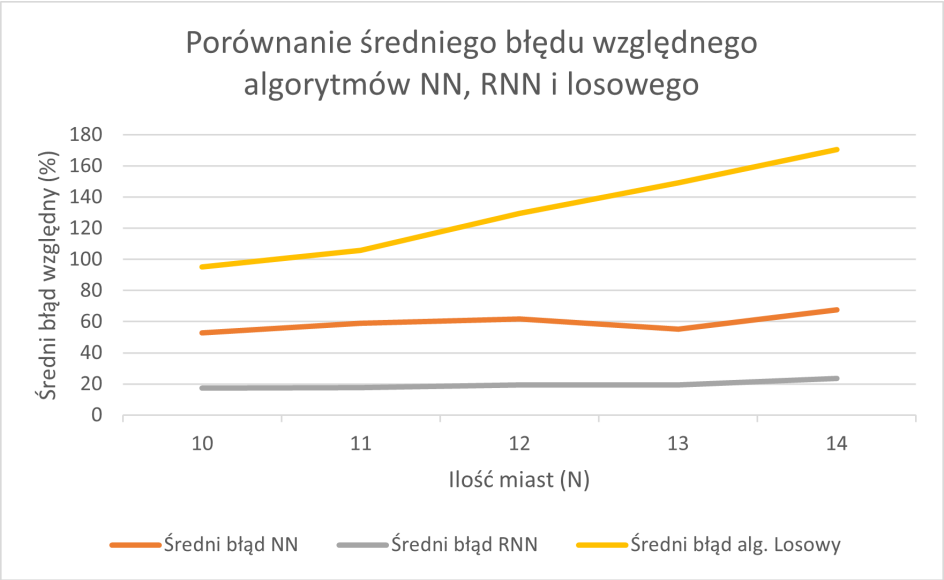

Figure 5: Porównanie średniego błędu względnego algorytmów NN, RNN i losowego

5 Wnioski

Z przeprowadzonych eksperymentów wynika, że czas wykonania algorytmu przeglądu zupełnego bardzo szybko rósł wraz ze wzrostem rozmiaru problemu przez co stawał się niepraktyczny dla większych rozmiarów macierzy. Spośród badanych algorytmów heurystycznych wyniki najbardziej zbliżone, obarczone najmniejszym błędem względem optymalnego rozwiązania zwracał algorytm RNN. Najgorzej wypadł algorytm losowy którego błąd dla stałej wartości permutacji wynoszącej 10N był znacznie większy i rosnął w znacznie szybszym tempie dla coraz to większych instancji niż w przypadku pozostałych dwóch algorytmów. Istnienie wielu ścieżek o tym samym koszcie optymalnym w grafie może negatywnie wpływać na czas wykonania algorytmów NN i RNN przez konieczność sprawdzania większej ilości tras, jednak zwiększa szansę uzyskania lepszego wyniku. Dla algorytmu Brute Force nie zmienia to

złożoności ponieważ w algorytmie tym i tak sprawdzane są wszystkie permutacje. Istotny wpływ na działanie algorytmów heurystycznych ma również zakres wag krawędzi. Mniejszy zakres wartości sprzyja częstszemu występowaniu remisów, co może wpłynąć na poprawę uzyskanego wyniku natomiast większy zakres powoduje, że decyzje algorytmów są częściej jednoznaczne.

6 Literatura

[1] Travelling salesman problem, Wikipedia: `https://en.wikipedia.org/wiki/Travelling_salesman_problem`, dostęp: 23.03.2026.
[2] math.hawaii.edu: `https://math.hawaii.edu/~les/m100/lecture9.pdf`, dostęp: 23.03.2026.
[3] Nearest neighbour algorithm, Wikipedia: `https://en.wikipedia.org/wiki/Nearest_neighbour_algorithm`, dostęp: 23.03.2026.